



Stingray™ PS225

SmartNIC Configuration Guide

Application Note

Broadcom, the pulse logo, Connecting everything, Stingray, Avago Technologies, Avago, and the A logo are among the trademarks of Broadcom and/or its affiliates in the United States, certain other countries, and/or the EU.

Copyright © 2018-2020 Broadcom. All Rights Reserved.

The term “Broadcom” refers to Broadcom Inc. and/or its subsidiaries. For more information, please visit www.broadcom.com.

Broadcom reserves the right to make changes without further notice to any products or data herein to improve reliability, function, or design. Information furnished by Broadcom is believed to be accurate and reliable. However, Broadcom does not assume any liability arising out of the application or use of this information, nor the application or use of any product or circuit described herein, neither does it convey any license under its patent rights nor the rights of others.

Table of Contents

1 Purpose and Audience	4
2 Introduction	4
3 Geographical Numbering of Hosts, Physical Functions, and Virtual Functions	5
4 Commands for Pairing	5
4.1 Create SmartNIC VF Pairs	5
4.2 Create SmartNIC VF Representor Pairs	6
4.3 Create SmartNIC PF Pairs	7
4.3.1 Pair Delete	7
4.3.2 Pair Query	7
5 DPDK Forwarding	8
5.1 SmartNIC VF Pairs	8
5.1.1 x86 Side Configuration	9
5.1.2 MAIA Side Configuration	10
6 OVS (Open vSwitch) Offload Models	12
6.1 SmartNIC VF Pairing Model	13
6.1.1 x86 Side Configuration	14
6.1.2 MAIA Side Configuration	15
6.2 SmartNIC Representor Pairing Model (with VxLAN tunnel)	17
6.2.1 x86 Side Configuration	18
6.2.2 MAIA Side Configuration	18
7 Miscellaneous – x86 Host Setup Information	22
7.1 Creating a VM	22
7.2 L2 Kernel Driver	22
7.3 DPDK	22
Appendix A: Acronyms and Abbreviations	23
Revision History	24
5880X-PS225-AN102; May 5, 2020	24
5880X-PS225-AN101; December 14, 2018	24
5880X-PS225-AN100; October 9, 2018	24

1 Purpose and Audience

This document provides background and information to enable setup and configuration of Stingray™ SmartNICs, including enabling capabilities such as OVS and DPDK, for dataplane processing and switching.

2 Introduction

This document is focused on SmartNIC solutions using Broadcom's Stingray BCM5880X System on a Chip (SoC), and includes details on the software infrastructure developed for SmartNIC use cases.

The Stingray BCM5880X device integrates a high-performance enterprise-class Ethernet controller (Nitro) and a powerful octal-core ARM Cortex-A72 processing subsystem. In SmartNIC applications, the BCM5880X appears and connects as a multi-function SR-IOV capable PCIe NIC endpoint to one or more host systems, typically server class x86 systems. Hosts may be based on Linux, FreeBSD, Windows, or VMWare and can make use of standard L2 kernel drivers to provide host Ethernet networking support. In addition, the host may use the standard DPDK poll mode driver to provide high-performance userspace-based Ethernet networking for network function virtualization (NFV) type applications.

For SmartNIC use cases, the OS running on the Stingray ARM complex is Linux-based. The integrated Ethernet controller (Nitro) also appears and connects as a multifunction SR-IOV capable PCIe NIC endpoint to the ARM complex (SoC host). Standard L2 drivers provide both kernel and userspace-based Ethernet Networking to the embedded SoC.

3 Geographical Numbering of Hosts, Physical Functions, and Virtual Functions

A numbering scheme is required to unambiguously refer to a specific PF or VF on a Host, including the SoC. The geographical numbering logically identifies the hosts as follows:

- 0 – A72 host
- 1 to 4 – x86 hosts 1 to 4

Physical functions are indexed globally with the first PF on host 0 as 0. PF index on other hosts starts at total number of PFs on all hosts with host number smaller than that host. For example, assuming host 0 has 8 PFs and host 1 has 3 PFs, host 0 PFs will be indexed as 0 to 7, and host 1 PFs is indexed as 8 to 10. Virtual functions are indexed logically, relative to the specified physical function, starting at index 0¹. The PFs are split between the two ports. The even-numbered PFs are assigned to port 0 the odd-numbered PFs to port 1.

4 Commands for Pairing

The `bnxt-ctl` tool is used to create pairs. The following `bnxt-ctl` commands are for the SmartNIC pairing model:

- `add-vf2fn-pair` – SmartNIC interface pairing command for VF-PF pair or VF-VF pair.
- `add-pf-pair` – SmartNIC interface pairing command for PF-PF pair.
- `add-rep2fn-pair` – SmartNIC representor pairing command.
- `del-pair` – Used to delete any of the above instantiated pairs.
- `show-pair` – Used to show any of the above instantiated pairs.

4.1 Create SmartNIC VF Pairs

The following command is used to pair VF functions with another VF or PF across multiple hosts:

```
bnxt-ctl add-vf2fn-pair <interface> [name] vf <index> [host <index>] [pf <index>] [vf <index>]
```

The `add-vf2fn-pair` command will pair a VF on the local host with a PF or VF on any host (including the local host). The command takes the following parameters:

- `<interface>` – The name of the PF interface that is the parent of the local VF interface to be paired. When the Linux kernel driver is bound to the PF, this may be the Linux name of the Ethernet interface associated with the PF (for example, `ethX`).
- `[name]` – An optional name of the multi-host pair. This will be used subsequently to reference the pair in other commands.
- `vf <index>` – The logical index of the local VF for pairing (relative to the parent PF).
- `host <index>` – The logical index of the host containing the partner interface.
- `pf <index>` – The global index of the PF on the host that is the partner PF interface, or the parent of the partner VF interface.
- `[vf <index>]` – The optional logical index of a VF that is the partner VF interface. If this option is omitted, the partner is a PF interface.

1. `Bnxt-ctl` design eventually changes so that physical functions are also indexed logically, relative to the specified host, starting at index 0. The change will be backward compatible with current design.

The following is a command line example that would be executed on the SoC to bind the fourth VF on a Broadcom Ethernet interface enP8p1s0f0np0 of the SoC with the first PF on the first x86 host, assuming there are eight PFs on the A72 host:

```
bnxt-ctl add-vf2fn-pair enP8p1s0f0np0 myPair vf 3 host 1 pf 8
```

The following is a command line example that would be executed on the SoC to bind the fourth VF on a Broadcom Ethernet interface enP8p1s0f0np0 of the SoC with the fifth VF on the first PF on host 1, assuming there are eight PFs on the A72 host:

```
bnxt-ctl add-vf2fn-pair enP8p1s0f0np0 myPair vf 3 host 1 pf 8 vf 4
```

4.2 Create SmartNIC VF Represor Pairs

The following command is required to pair a represor on the SoC with a PF or VF on the x86 host:

```
bnxt-ctl add-rep2fn-pair <interface> [name] [host <index>] [pf <index>] [vf <index>]
```

The add-rep2fn-pair command pairs a represor on the local host with a PF or VF on any host (including the local host). The command takes the following parameters:

- <interface> – The name of the PF interface that is the parent of the local VF interface to be paired. When the Linux kernel driver is bound to the PF, this may be the Linux name of the Ethernet interface associated with the PF (for example, ethX). Alternatively, this name may be specified using the PCIe<domain>:<bus>:<slot>.<function> string of the PF (for example, 0000:05:00.1). The interface must be a Broadcom Ethernet interface owned by Linux even if PCIe naming is used.
- [name] – An optional name of the SmartNIC represor pair. This is used subsequently to reference the pair in other commands.
- host <index> – The logical index of the host containing the partner interface
- pf <index> – The global index of the PF on the host that is the partner PF interface, or the parent of the partner VF interface.
- [vf <index>] – The optional logical index of a VF that is the partner VF interface. If this option is omitted, the partner is a PF interface.

The following is a command line example that would be executed on the SoC to bind a named represor on a Broadcom Ethernet interface enP8p1s0f0np0 of the SoC with the first PF on the first x86 host, assuming there are eight PFs on A72 host:

```
bnxt-ctl add-rep2fn-pair enP8p1s0f0np0 rep0 host 1 pf 8
```

The following is a command line example that would be executed on the SoC to bind a named represor on a Broadcom Ethernet interface enP8p1s0f0np0 of the SoC with the fifth VF on the first PF on the first x86 host, assuming there are eight PFs on A72:

```
bnxt-ctl add-rep2fn-pair enP8p1s0f0np0 rep0 host 1 pf 8 vf 4
```

4.3 Create SmartNIC PF Pairs

The following command is used to pair PF functions with another PF across multiple hosts:

```
bnxt-ctl add-pf-pair <interface> [name] host <index> pf <index>
```

The add-pf-pair command pairs a PF on the local host with a PF on any host (including the local host). The command takes the following parameters:

- <interface> – The name of the local PF interface to be paired. When the Linux kernel driver is bound to the PF, this may be the Linux name of the Ethernet interface associated with the PF (for example, ethX). Alternatively, this name may be specified using the PCIe<domain>:<bus>:<slot>.<function> string of the PF (for example, 0000:05:00.1). The interface must be a Broadcom Ethernet interface owned by Linux even if PCIe naming is used.
- [name] – A name of the PF pair. This is used subsequently to reference the pair in other commands.
- host <index> – The logical index of the host containing the partner interface.
- pf <index> – The global index of the PF on the host that is the partner PF interface, or the parent of the partner VF interface.

The following is a command line example that would be executed on the SoC to bind a named representor on a Broadcom Ethernet interface enP8p1s0f0np0 of the SoC with the first PF on the first x86 host, assuming there are eight PFs on A72:

```
bnxt-ctl add-pf-pair enP8p1s0f0np0 pfpair0 host 1 pf 8
```

4.3.1 Pair Delete

The following command is used to delete interface pairs, representor pairs, or PF pairs:

```
bnxt-ctl del-pair <interface> <name>
```

The command takes the following parameters:

- <interface> – The name of a Broadcom Ethernet interface. The interface is only used by bnxt-ctl to communicate with Chimp firmware. It does not have to be one of the paired interface.
- [name] – Pair name used for pair creation.

4.3.2 Pair Query

The following command is used to get information and statistics of interface pairs, representor pairs or PF pairs:

```
bnxt-ctl show-pair <interface> <name>
```

The command takes following parameters:

- <interface> – The optional name of a Broadcom Ethernet interface. If the interface is not specified, bnxt-ctl goes through all valid Broadcom Ethernet interfaces owned by Linux and display all pairs on those interfaces. Pairs are not displayed if the endpoint interface is no longer owned by Linux, for example, and transferred to DPDK.
- <name> – Optional parameter that is valid only when <interface> is specified. Specify pair name used for pair creation. If no name is specified, display all pairs with the specified interface as one end point.

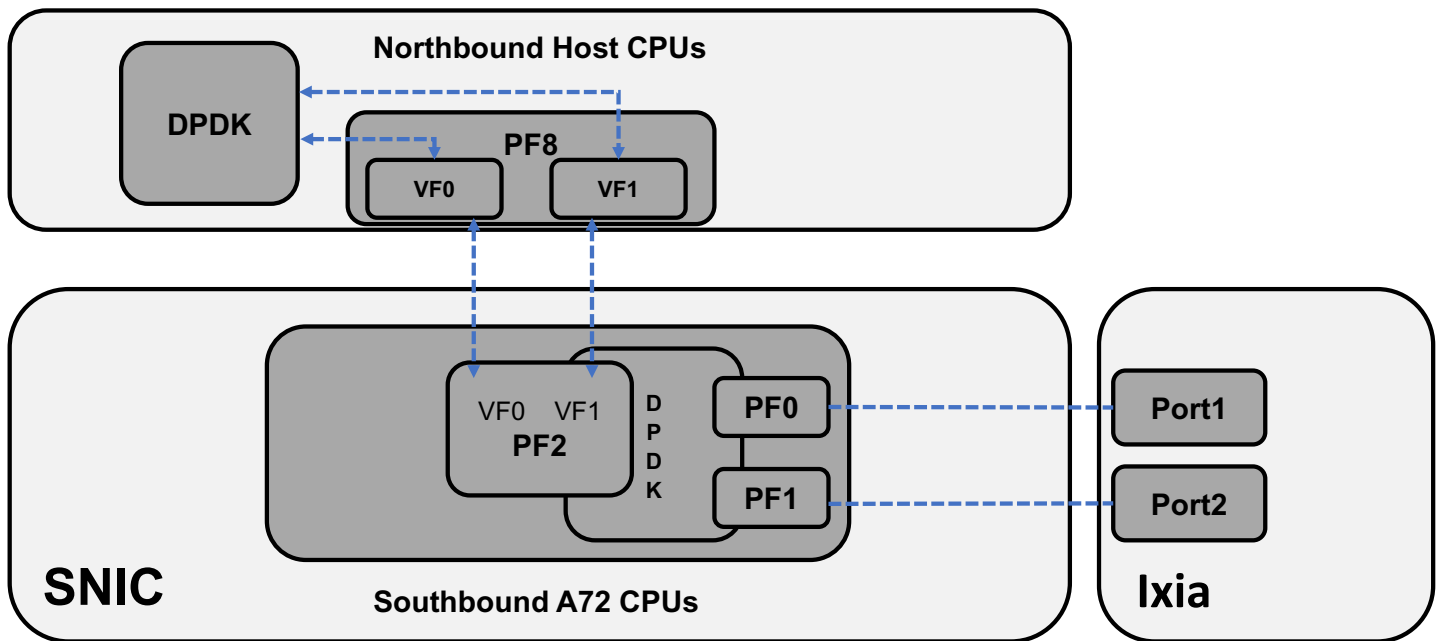
5 DPDK Forwarding

This section describes the configuration steps required to test the DPDK (testpmd) application running on host (x86) and on Maia (A72) with the VF pair setup.

NOTE: Paths and directory names can be bit different based on where the software components are installed and the version of the packages being installed and used.

5.1 SmartNIC VF Pairs

Figure 1: Packet Flow Path



Packet flow path: IXIA port1 → PF0 → DPDK → VF0(PF2) on a MAIA pair (VF0(PF8) on an x86) → DPDK → VF1(PF8) on an x86 pair (VF1(PF2) on a MAIA) → DPDK → PF1 → IXIA port2.

5.1.1 x86 Side Configuration

To configure SmartNIC VF pairs on an x86:

1. Install `bnxt_en.ko` with the following commands:

```
modprobe vxlan
insmod bnxt_en.ko
```

2. Bring up PF8 (the first PF on the host) with the following command:

```
ifconfig enp3s0f0 up
```

3. Create VFs on the PF8 with the following command:

```
echo 2 > /sys/class/net/enp3s0f0/device/sriov_numvfs
```

4. Check the driver bind status with the following command:

```
/root/dpdk/tools/dpdk-devbind.py --status
```

5. Install the dpdk driver and bind the interfaces (here VF interfaces are used) on the DPDK that have to run to the driver with the following commands:

```
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge
echo 1024 > /sys/devices/system/node/node0/hugepages/hugepages-2048kB/nr_hugepages
modprobe uio
insmod /root/dpdk/build/kmod/igb_uio.ko
```

To list and discover the PCIe bus, device, and function for the SmartNIC use command `lspci -n` and use the information with the `dpdk-devbind.py` command:

NOTE: In this example, BDF (Bus, Device, Function) is based on the SmartNIC connected to PCIe slot on the system that has been used.

```
/root/dpdk/usertools/dpdk-devbind.py --bind=igb_uio 04:00.0
/root/dpdk/usertools/dpdk-devbind.py --bind=igb_uio 04:00.1
```

```
/root/dpdk/usertools/dpdk-devbind.py --status
```

Ensure that the interfaces are bonded to the `igb_uio` driver.

6. Run `testpmd` and start `testpmd` with the following commands:

```
/root/dpdk/build/app/testpmd -c 0xf -n 2 -- --total-num-mbufs=40960 -i --rxq=1 --txq=1 \
--rxd=2048 --txd=2048 --port-topology=paired
testpmd> start
testpmd> show port stats all (shows the port stats)
```

5.1.2 MAIA Side Configuration

To configure SmartNIC VF pairs on MAIA:

1. If `bnxt_en.ko` is not installed when the system boots to Linux, install it.
2. Check the list of PFs (`ifconfig -a`) and bring up the PFs needed with the following commands:

NOTE: The following list is supported by the NXS SN 1.1.9.2 release.

```
ifconfig enP8p1s0f0np0 up
ifconfig enP8p1s0f1np1 up
ifconfig enP8p1s0f2np0 up
```

3. Create VFs with the following command:

```
echo 2 > /sys/class/net/enP8p1s0f2np0/device/sriov_numvfs
```

4. Create interface pairs with the following commands:

```
/usr/bin/bnxt-ctl add-vf2fn-pair enP8p1s0f2np0 vfpair0 vf 0 host 0 pf 8 vf 0
/usr/bin/bnxt-ctl add-vf2fn-pair enP8p1s0f2np0 vfpair1 vf 1 host 0 pf 8 vf 1
```

5. Install dpdk driver and bind the interfaces (the first 2 PFs and 2 VFs created on the third PF are used) on which DPDK has to run to the driver with the following commands:

```
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
modprobe uio
insmod /usr/share/dpdk/arm64-stingray-linuxapp-gcc/igb_uio.ko
/usr/share/dpdk/usertools/dpdk-devbind.py --status

echo 0008:01:00.0 >/sys/bus/pci/drivers/bnxt_en/unbind
echo 0008:01:00.1 >/sys/bus/pci/drivers/bnxt_en/unbind
echo 0008:01:03.0 >/sys/bus/pci/drivers/bnxt_en/unbind
echo 0008:01:03.1 >/sys/bus/pci/drivers/bnxt_en/unbind
/usr/share/dpdk/usertools/dpdk-devbind.py --status

echo "14e4 d802" >/sys/bus/pci/drivers/igb_uio/new_id
echo "14e4 d800" >/sys/bus/pci/drivers/igb_uio/new_id
```

NOTE: 14e4 d802 is the PF ID and 14e4 d800 is the VF ID (this comes from `lspci -n`).

```
/usr/share/dpdk/usertools/dpdk-devbind.py --status
```

NOTE: Retrieve the DPDK from the tarball provided by the Broadcom representative. Do not download the DPDK from an internet source that does not support ID =d800.

The system displays the following:

```
root@bcm958802a8020:~# /usr/share/dpdk/usertools/dpdk-devbind.py --status
```

```
Network devices using DPDK-compatible driver
```

```
=====
```

```
0008:01:00.0 'Device d802' drv=igb_uio unused=bnxt_en,vfio-pci
0008:01:00.1 'Device d802' drv=igb_uio unused=bnxt_en,vfio-pci
0008:01:03.0 'Device 16f4' drv=igb_uio unused=bnxt_en,vfio-pci
0008:01:03.1 'Device 16f4' drv=igb_uio unused=bnxt_en,vfio-pci
```

```
Network devices using kernel driver
```

```
=====
```

```
0008:01:00.2 'Device d802' if=enP8p1s0f2np0 drv=bnxt_en unused=igb_uio,vfio-pci
0008:01:00.3 'Device d802' if=enP8p1s0f3np1 drv=bnxt_en unused=igb_uio,vfio-pci
0008:01:00.4 'Device d802' if=enP8p1s0f4np0 drv=bnxt_en unused=igb_uio,vfio-pci
0008:01:00.5 'Device d802' if=enP8p1s0f5np1 drv=bnxt_en unused=igb_uio,vfio-pci
0008:01:00.6 'Device d802' if=enP8p1s0f6np0 drv=bnxt_en unused=igb_uio,vfio-pci
0008:01:00.7 'Device d802' if=enP8p1s0f7np1 drv=bnxt_en unused=igb_uio,vfio-pci
```

```
Other network devices
```

```
=====
```

```
<none>
```

```
Crypto devices using DPDK-compatible driver
```

```
=====
```

```
<none>
```

```
Crypto devices using kernel driver
```

```
=====
```

```
<none>
```

```
Other crypto devices
```

```
=====
```

```
<none>
```

6. Run testpmd and start testpmd with the following command:

```
/usr/bin/testpmd -c 0xfe -n 7 -- --total-num-mbufs=40960 -i --rxq=3 --txq=3 \
--rxd=256 --txd=256 --port-topology=paired
testpmd> set portlist 0,2,1,3 (set the ports, example)
testpmd> set corelist 2,3,4,5,6,7 (set the cores to be used, cores 2,3,4,5,6,7 are used here,
example)
previous number of forwarding cores 1 - changed to number of configured cores 6
testpmd> start (start testpmd)
testpmd> show port stats all (shows the port stats)
```

NOTE: testpmd usage guide: http://dpdk.org/doc/guides/testpmd_app_ug/testpmd_funcs.html

-c 0xfe above is the CPU mask, binary- 1111 1110, so 7 cores are used (specify the cores to be used using set corelist), change according to your test requirements).

Based on your configuration, send a packet from one port on the link partner (IXIA or something else) and another port on the link partner receives the packet.

Other testpmd commands to stop and quit:

```
testpmd> stop (stop testpmd)
testpmd> quit (quit testpmd)
```

6 OVS (Open vSwitch) Offload Models

This section describes the offload models that PS225 SmartNIC supports, specifically the userspace (using DPDK) offload models related to the OVS datapath.

The two interface pairing models supported with the PS225 SmartNIC are as follows:

- VF pairing
- Representer pairing

For reference, this document describes both VF pair and VF Representer pair configuration steps.

SmartNIC VF Pairing Model

SmartNIC supports a VF pair model. With the VF pair configuration, the host can support up to 64 VFs which VMs or an application uses one or more VF(s) each.

SmartNIC Representer Pairing Model

On a SmartNIC(PS225), to extend the scale beyond 64 interface pairs (up to 128 pairs), SmartNIC Representer Pairing Model can be used. For software switching, a single SoC application may terminate a large scale of paired interfaces. The SmartNIC Representer Pairing Model enables a single PF on the SoC to demultiplex and multiplex multiple pairing interfaces which are paired with functions (VFs and/or PFs) on the x86 hosts.

NOTE: The VF pairing model is limited by the MAIA side 64 VF limitation. The host (x86) can create up to 128 VFs.

On SmartNIC(PS225), Nitro must be configured with a relevant PF profile. See [Table 1](#) for information on different Nitro PF profiles.

Table 1: Nitro PF Profiles

PF Profile Nitro Config File	Description
bcm958802a8048_2xAN_8+3_pf.pkg	Eight PFs on MAIA (A72) side and three PFs on Host(x86) side for VF representer pair configuration support.
bcm958802a8048_2xAN_8+4_pf.pkg	Eight PFs on MAIA (A72) side and four PFs on Host(x86) side for VF pair configuration support.
bcm958802a8048_2xAN_8+4_pf_rep.pkg	Eight PFs on MAIA (A72) side and four PFs on Host(x86) side for VF representer pair configuration support.

These .pkg files are in the nitro/extra_cfgs/ directory of the image tarball.

To load the .pkg (for example, bcm958802a8048_2xAN_8+4_pf.pkg) file:

1. On MAIA (A72 cores) console:

```
update-me.sh -s 192.168.1.20 -d nic -t tftp -i nitro -c bcm958802a8048_2xAN_8+4_pf
```

2. Reboot or power cycle the system.

6.1 SmartNIC VF Pairing Model

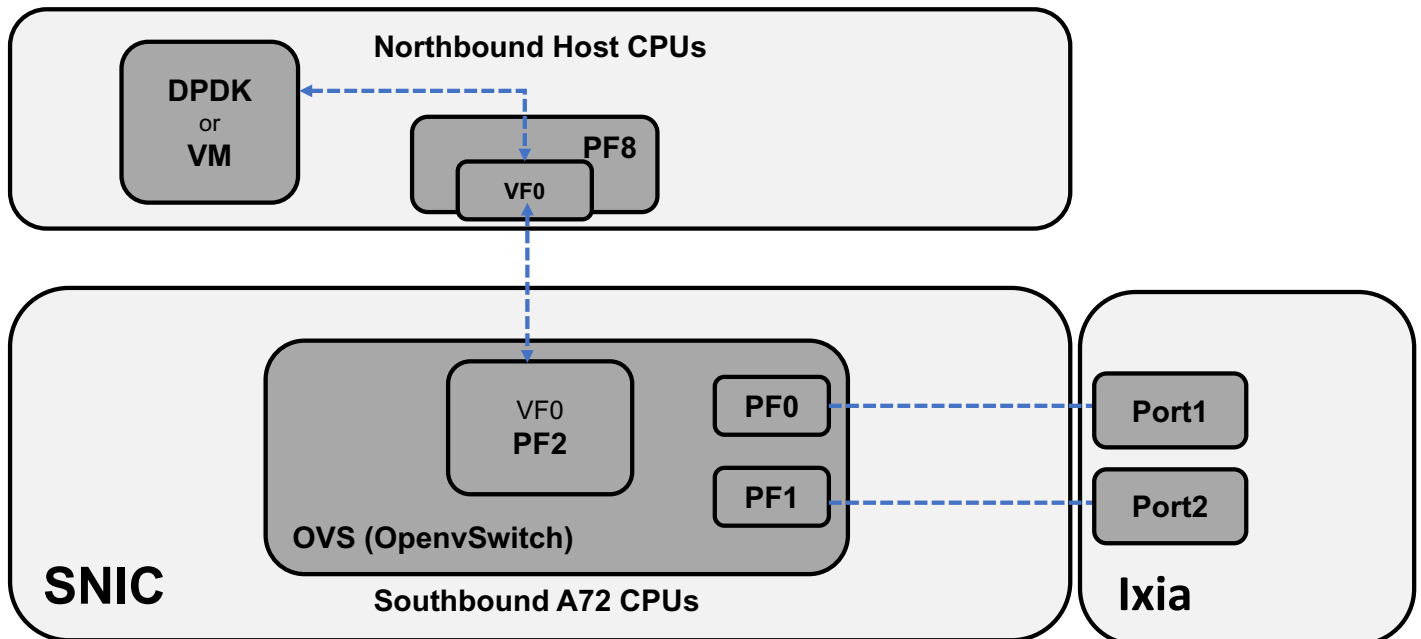
This section describes the configuration steps to test the OVS + DPDK application running on MAIA (A72) with VF pairs set up between the host (x86) and MAIA (A72).

The following .pkg file should be programmed from the MAIA console:

```
update-me.sh -s 192.168.1.20 -d nic -t tftp -i nitro -c bcm958802a8048_2xAN_8+4_pf.pkg
```

NOTE: The configuration steps in this section are supported by the NXS_SN 1.1.9.2 release. The interface naming conventions change with the version of software used for the configuration steps and tests.

Figure 2: Setup for VF Pairs



Packet flow path: IXIA port2 → PF1 → OVS → VF0 (PF2) on MAIA pair (VF0 (PF8) on x86 (DPDK) pair) → VF0 (PF2) on MAIA → OVS → PF0 → IXIA port1.

- PF0, PF1, VF (on PF2) on MAIA added to OVS bridge.
- DPDK on VF (on PF8) on x86.

6.1.1 x86 Side Configuration

To configure SmartNIC VF pairs on an x86:

1. Install `bnxt_en.ko` with the following commands:

```
modprobe vxlan
insmod bnxt_en.ko
```

2. Bring up PF8 (first PF on the host) with the following command:

```
ifconfig enp3s0f0 up
```

3. Disable IPv6 as the PFs keep sending ICMPv6 packets with the following command:

```
echo 1 > /proc/sys/net/ipv6/conf/all/disable_ipv6
```

4. Create VFs on the PF8 with the following command:

```
echo 1 > /sys/class/net/enp3s0f0/device/sriov_numvfs
ifconfig enp4s0 up
```

5. Check the driver bind status with the following command:

```
/root/dpdk/usertools/dpdk-devbind.py --status
```

6. Install the DPDK driver and bind the interfaces (VF interfaces are used) on which DPDK has to run to the driver with the following commands:

```
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge
echo 1024 > /sys/devices/system/node/node0/hugepages/hugepages-2048kB/nr_hugepages
modprobe uio
insmod /root/dpdk/build/kmod/igb_uio.ko
lspci -n
/root/dpdk/usertools/dpdk-devbind.py --bind=igb_uio 04:00.0
```

7. Ensure that the interfaces are bonded to the `igb_uio` driver. Run `testpmd` and start `testpmd` using the following commands:

```
/root/dpdk/build/app/testpmd -c 0xf -n 2 -- --total-num-mbufs=40960 -i --rxq=1 --txq=1 \
--rxd=2048 --txd=2048
testpmd> set corelist 2,3
testpmd> start
testpmd> show port stats all
```

6.1.2 MAIA Side Configuration

To configure SmartNIC VF pairs on MAIA:

1. Check the list of PFs (ifconfig -a) and bring up the PFs required with the following commands:

NOTE: The following list is with release NXS_SN 1.1.9.2.

```
ifconfig enP8p1s0f0np0 up
ifconfig enP8p1s0f1np1 up
ifconfig enP8p1s0f2np0 up
```

2. Create the VF on PF2 with the following command:

```
echo 1 > /sys/class/net/enP8p1s0f2np0/device/sriov_numvfs
```

3. Bring up the VF interface with the following command:

```
ifconfig enP8p1s3 up
```

4. Create the VF interface pair with the following command:

```
/usr/bin/bnxt-ctl add-vf2fn-pair enP8p1s0f2np0 vfpair0 vf 0 host 1 pf 8 vf 0
```

5. Install the DPDK driver and bind the interfaces (the first two PFs and one VF created on the third PF) on which DPDK has to run to the driver with the following commands:

```
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
modprobe uio
insmod /usr/share/dpdk/arm64-stingray-linuxapp-gcc/igb_uio.ko
/usr/share/dpdk/usertools/dpdk-devbind.py --status
lspci -n
echo 0008:01:00.0 >/sys/bus/pci/drivers/bnxt_en/unbind
echo 0008:01:00.1 >/sys/bus/pci/drivers/bnxt_en/unbind
echo 0008:01:03.0 >/sys/bus/pci/drivers/bnxt_en/unbind
/usr/share/dpdk/usertools/dpdk-devbind.py --status

echo "14e4 d802" >/sys/bus/pci/drivers/igb_uio/new_id
echo "14e4 d800" >/sys/bus/pci/drivers/igb_uio/new_id

/usr/share/dpdk/usertools/dpdk-devbind.py --status

rm -rf /etc/openvswitch/
rm -rf /var/run/openvswitch
rm -rf /var/log/openvswitch
mkdir -p /etc/openvswitch
mkdir -p /var/run/openvswitch
mkdir -p /var/log/openvswitch
```

6. Create the OVS database with the following command:

```
/usr/bin/ovsdb-tool create /etc/openvswitch/conf.db /usr/share/openvswitch/vswitch.ovsschema
```

7. Start the OVSDb server with the following commands:

```
/usr/sbin/ovsdb-server --remote=punix:/var/run/openvswitch/db.sock \  
--remote=db:Open_vSwitch,Open_vSwitch,manager_options \  
--private-key=db:Open_vSwitch,SSL,private_key \  
--certificate=db:Open_vSwitch,SSL,certificate \  
--bootstrap-ca-cert=db:Open_vSwitch,SSL,ca_cert \  
--pidfile --detach --log-file  
  
/usr/bin/ovs-vsctl --no-wait set Open_vSwitch . other_config:dppk-init=true  
  
/usr/sbin/ovs-vswitchd unix:/var/run/openvswitch/db.sock --pidfile --detach --log-file  
  
/usr/bin/ovs-vsctl --no-wait set Open_vSwitch . other_config:dppk-socket-mem="1024,0"
```

8. Create or add the bridge with the following command:

```
/usr/bin/ovs-vsctl add-br br0 -- set bridge br0 datapath_type=netdev
```

9. Set the CPU mask (number of CPU cores) to run OVS on seven cores out of a MAIA eight core with the following command:

```
/usr/bin/ovs-vsctl set Open_vSwitch . other_config:pmd-cpu-mask=0xfe
```

10. Add the DPDK ports (PF0, PF1, VF0(PF2)) into the OVS bridge with the following commands:

```
/usr/bin/ovs-vsctl add-port br0 dpdk0 -- set Interface dpdk0 type=dpdk \  
options:dpdk-devargs=0008:01:00.0 ofport_request=1  
  
/usr/bin/ovs-vsctl add-port br0 dpdk1 -- set Interface dpdk1 type=dpdk \  
options:dpdk-devargs=0008:01:00.1 ofport_request=2  
  
/usr/bin/ovs-vsctl add-port br0 dpdk2 -- set Interface dpdk2 type=dpdk \  
options:dpdk-devargs=0008:01:03.0 ofport_request=3  
  
ifconfig br0 up
```

11. List the bridge ports with the following command:

```
/usr/bin/ovs-vsctl list-ports br0
```

12. Set the bridge flows with the following commands:

```
/usr/bin/ovs-ofctl add-flow br0 in_port=2,action=output:3  
/usr/bin/ovs-ofctl add-flow br0 in_port=3,action=output:1
```

13. Set the queues to the ports with the following commands:

```
/usr/bin/ovs-vsctl set Interface dpdk0 options:n_rxq=1  
/usr/bin/ovs-vsctl set Interface dpdk1 options:n_rxq=3  
/usr/bin/ovs-vsctl set Interface dpdk2 options:n_rxq=3
```


14. List the interfaces with the following commands:

```
/usr/bin/ovs-vsctl list interface dpdk0
/usr/bin/ovs-vsctl list interface dpdk1
/usr/bin/ovs-vsctl list interface dpdk2
```

```
ifconfig enP8p1s0f2np0 down
```

15. Dump the counters with the following commands:

```
/usr/bin/ovs-appctl bridge/dump-flows br0
/usr/bin/ovs-ofctl dump-flows br0
```

16. Display the VF pair counter data with the following command:

```
/usr/bin/bnxt-ctl show-pair enP8p1s0f2np0 vfpair0
```

6.2 SmartNIC Represor Pairing Model (with VxLAN tunnel)

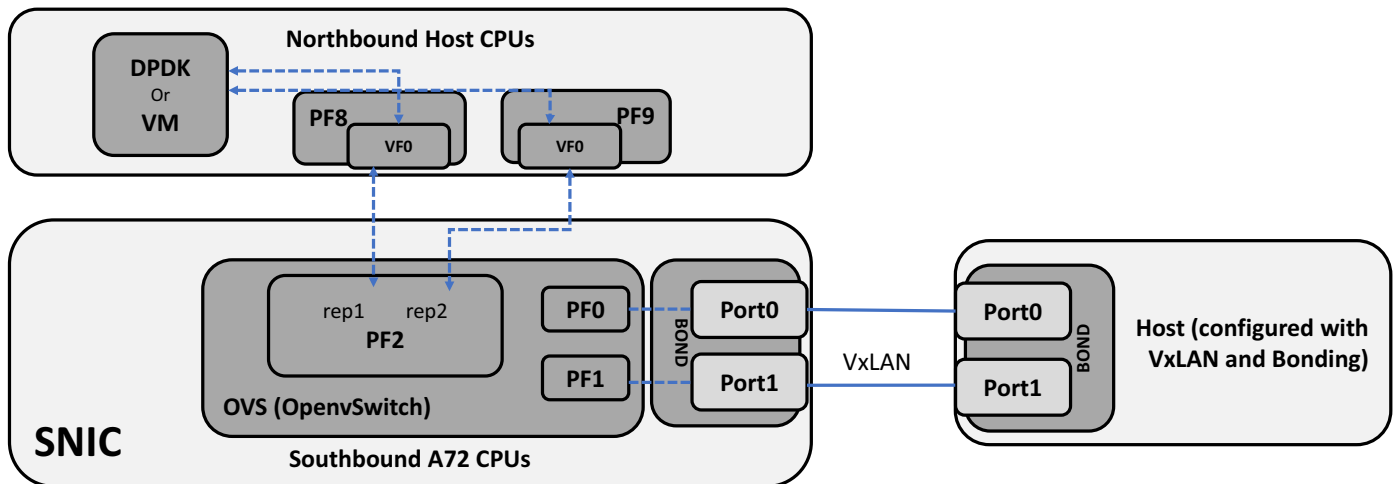
This section describes the configuration steps to test the OVS + DPDK application running on MAIA(A72) with a VF represor pair setup between a host (x86) and a MAIA (A72). This setup configuration is with the VxLAN tunnel protocol.

The following .pkg file should be programmed from the MAIA console:

```
update-me.sh -s 192.168.1.20 -d nic -t tftp -i nitro -c bcm958802a8048_2xAN_8+3_pf
```

NOTE: The interface naming conventions change with the version of software used for the configuration steps and tests.

Figure 3: Setup for VF Represor Pairs



The following configuration sets up network connectivity between the left host (via SmartNIC) and the right host. On the left host where the SmartNIC is installed, packet switching from physical network ports to VFs on the host is handled by OVS + DPDK through VF-represor pairs.

When VMs are used on the left host, the VF driver (L2 driver) must be loaded on the VMs and the VFs on the host are attached to the VMs.

6.2.1 x86 Side Configuration

To configure SmartNIC representor pairs on an x86:

1. Install `bnxt_en.ko` with the following commands:

```
modprobe vxlan
insmod bnxt_en.ko
```

2. Bring up the PF8 and PF9 (the first and second PF on the host) with the following commands:

```
ifconfig enp129s0f0 up
ifconfig enp129s0f1d1 up
```

3. Disable IPV6 since PFs keep sending ICMPv6 packets with the following command:

```
echo 1 > /proc/sys/net/ipv6/conf/all/disable_ipv6
```

4. Create VFs on the PF8 and PF9 with the following commands:

```
echo 1 > /sys/class/net/enp129s0f0/device/sriov_numvfs
echo 1 > /sys/class/net/enp129s0f1d1/device/sriov_numvfs
```

5. Bring the VF interfaces up with the following commands:

```
ifconfig enp130s0 up
ifconfig enp130s8 up
```

6. Configure interface PF10 (third PF on the host) for the control channel between the host and A72 (MAIA) with the following command:

```
ifconfig enp129s0f2 192.18.1.2 netmask 255.255.255.0 up
```

7. Configure the VF interface mac(ether) and IP with the following commands:

```
ifconfig enp130s0 hw ether 00:00:11:11:00:01
ifconfig enp130s0 11.11.0.1 netmask 255.255.255.0 up
sleep 1
ifconfig enp130s8 hw ether 00:00:11:11:40:01
ifconfig enp130s8 11.11.64.1 netmask 255.255.255.0 up
```

6.2.2 MAIA Side Configuration

To configure SmartNIC representor pairs on MAIA:

1. Check the list of PFs (`ifconfig -a`) and bring up the PFs required with the following commands:

```
ifconfig enP8p1s0f0np0 up
ifconfig enP8p1s0f1np1 up
ifconfig enP8p1s0f2np0 up
```

2. Configure the control channel using PF3 with the following commands:

```
ifconfig enP8p1s0f3np1 up
ifconfig enP8p1s0f3np1 192.18.1.1
```

```
bnxt-ctl del-pair enP8p1s0f3np1 pfpair-ctrl-chnl
bnxt-ctl add-pf-pair enP8p1s0f3np1 pfpair-ctrl-chnl host 1 pf 10 q_ab 2 q_ba 3 fc_ab 0 fc_ba 0
```

3. Configure the representor pairs with the following commands:

```
bnxt-ctl add-rep2fn-pair enP8p1s0f2np0 0000:06:00.0 host 1 pf 8 vf 0
bnxt-ctl add-rep2fn-pair enP8p1s0f2np0 0000:06:00.40 host 1 pf 9 vf 0
```

Configuring DPDK

To configure DPDK:

1. Install the DPDK driver and bind the interfaces (All three PFs) on which DPDK must run to the driver with the following commands:

```
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages

insmod /usr/share/dpdk/arm64-stingray-linuxapp-gcc/igb_uio.ko

/usr/share/dpdk/usertools/dpdk-devbind.py --force --bind igb_uio 0008:01:00.0
/usr/share/dpdk/usertools/dpdk-devbind.py --force --bind igb_uio 0008:01:00.1
/usr/share/dpdk/usertools/dpdk-devbind.py --force --bind igb_uio 0008:01:00.2
```

2. Configure OVS with the following commands:

```
ovs-appctl -t ovsdb-server exit
ovs-appctl -t ovs-vswitchd exit

rm -rf /etc/openvswitch/
rm -rf /var/run/openvswitch
rm -rf /var/log/openvswitch

mkdir -p /etc/openvswitch
mkdir -p /var/run/openvswitch
mkdir -p /var/log/openvswitch
```

3. To create the OVS database, use the following command:

```
/usr/bin/ovsdb-tool create /etc/openvswitch/conf.db /usr/share/openvswitch/vswitch.ovsschema
```

4. To start the OVSDb server, use the following commands:

```
/usr/sbin/ovsdb-server --remote=punix:/var/run/openvswitch/db.sock \
--remote=db:Open_vSwitch,Open_vSwitch,manager_options \
--private-key=db:Open_vSwitch,SSL,private_key \
--certificate=db:Open_vSwitch,SSL,certificate \
--bootstrap-ca-cert=db:Open_vSwitch,SSL,ca_cert \
--pidfile --detach --log-file

ovs-vsctl --no-wait set Open_vSwitch . other_config:dpdk-init=true

/usr/sbin/ovs-vswitchd unix:/var/run/openvswitch/db.sock --pidfile --detach --log-file

ovs-vsctl --no-wait set Open_vSwitch . other_config:dpdk-socket-mem="1024,0"
```

5. Configure the bridges with the following commands:

```
ovs-vsctl --if-exists del-br br-int
ovs-vsctl --if-exists del-br br-ext
ovs-vsctl add-br br-int -- set bridge br-int datapath_type=netdev datapath_type=netdev \
-- br-set-external-id br-int bridge-id br-int -- set bridge br-int fail-mode=standalone
ovs-vsctl add-br br-ext -- set bridge br-ext datapath_type=netdev datapath_type=netdev \
-- br-set-external-id br-ext bridge-id br-ext -- set bridge br-ext fail-mode=standalone
```

6. To enable the dpdk PMD CPU mask, use the following command:

```
ovs-vsctl set Open_vSwitch . other_config:pmd-cpu-mask=0xfe
```

7. Configure the DPDK bond port with the following commands:

```
ovs-vsctl --if-exists del-port br-ext bond0

ovs-vsctl add-bond br-ext bond0 dpdk129 dpdk130 -- set Interface dpdk129 type=dpdk \
options:dpdk-devargs=0008:01:00.0 mtu_request=1900 options:n_rxq_desc=256 \
options:n_txq_desc=256 -- set Interface dpdk130 type=dpdk \
options:dpdk-devargs=0008:01:00.1 mtu_request=1900 options:n_rxq_desc=256 \
options:n_txq_desc=256

ovs-vsctl set interface dpdk129 options:n_rxq=6 \
other_config:pmd-rxq-affinity=0:1,1:2,2:3,3:4,4:5,5:6
ovs-vsctl set interface dpdk130 options:n_rxq=6 \
other_config:pmd-rxq-affinity=0:1,1:2,2:3,3:4,4:5,5:6

ovs-vsctl set port bond0 bond_mode=balance-tcp
ovs-vsctl set port bond0 lacp=active
ovs-appctl bond/show
```

8. Configure the representor ports with the following commands:

#VNI based forwarding:

```
ovs-ofctl del-flows br-int
ovs-vsctl --if-exists del-port br-int dpdk1
ovs-vsctl add-port br-int dpdk1 -- set Interface dpdk1 \
type=dpdk options:dpdk-devargs=0000:06:00.0 mtu_request=1900 \
options:n_rxq_desc=256 options:n_txq_desc=256 ofport_request=1

ovs-vsctl set interface dpdk1 options:n_rxq=1 other_config:pmd-rxq-affinity=0:7

ovs-ofctl add-flow br-int 'table=0,in_port=1,actions=set_field:1->tun_id,resubmit(,1)'
ovs-ofctl add-flow br-int table=1,in_port=1000,tun_id=1,action=output:1
ovs-ofctl add-flow br-int 'table=0,in_port=65,actions=set_field:65->tun_id,resubmit(,1)'
ovs-ofctl add-flow br-int table=1,in_port=1000,tun_id=65,action=output:65

ovs-vsctl --if-exists del-port br-int dpdk65

ovs-vsctl add-port br-int dpdk65 -- set Interface dpdk65 \
type=dpdk options:dpdk-devargs=0000:06:00.40 mtu_request=1900 \
options:n_rxq_desc=256 options:n_txq_desc=256 ofport_request=65

ovs-vsctl set interface dpdk65 options:n_rxq=1 other_config:pmd-rxq-affinity=0:7
```

```
ovs-ofctl add-flow br-int 'table=0,in_port=65,actions=set_field:65->tun_id,resubmit(,1)'
```

```
ovs-ofctl add-flow br-int table=1,in_port=1000,tun_id=65,action=output:65
```

9. Configure the default flow rules with the following commands:

```
ovs-ofctl add-flow br-int 'table=0,actions=resubmit(,1)'
```

```
ovs-ofctl add-flow br-int table=1,actions=output:1000
```

```
ovs-vsctl --if-exists del-port br-int dpdk133
```

```
ovs-vsctl add-port br-int dpdk133 -- set Interface dpdk133 \
type=dpdk options:dpdk-devargs=0008:01:00.2 mtu_request=1900 \
options:n_rxq_desc=256 options:n_txq_desc=256 ofport_request=133
```

```
ovs-vsctl set interface dpdk133 options:n_rxq=6 \
other_config:pmd-rxq-affinity=0:1,1:2,2:3,3:4,4:5,5:6
```

10. Configure the VXLAN with the following commands:

```
ovs-vsctl add-port br-int vxlan0 -- set interface vxlan0 type=vxlan \
options:{remote_ip="192.168.18.2",local_ip="192.168.18.1", \
in_key=flow,out_key=flow} ofport_request=1000
```

```
ifconfig br-ext 192.168.18.1
```

```
ovs-appctl ovs/route/add 192.168.18.2/24 br-ext
```

11. List the bridge ports with the following commands:

```
ovs-vsctl list-ports br-int
```

```
ovs-vsctl list-ports br-ext
```

12. List the interfaces with the following command:

```
/usr/bin/ovs-vsctl list interface dpdk1
```

13. Dump the counters and flows with the following commands:

```
ovs-ofctl dump-ports br-int
```

```
ovs-ofctl dump-flows br-int
```

```
/usr/bin/ovs-appctl bridge/dump-flows br-int
```

```
/usr/bin/ovs-appctl bridge/dump-flows br-ext
```

14. Display the control pair PF interface counter data with the following command:

```
bnxt-ctl show-pair enP8p1s0f3np1
```

This configuration creates a VF representor pair model setup with OVS running on SmartNIC ARM cores to accomplish networking traffic from the host VMs (or any other traffic processing application) to the peer system over VxLAN tunneling with port bonding. The peer system can be any network entity supporting VxLAN and port bonding.

Additionally, the peer system can be replicated using same as left side setup shown in [Figure 3](#). Ensure that you configure the correct VxLAN tunnel endpoint local and remote IP addresses during the configuration steps.

NOTE: The VF representor pairing model can be setup without the use of VxLAN and bonding as well. It requires some of the configuration steps to be modified to remove the VxLAN and bonding setup.

7 Miscellaneous – x86 Host Setup Information

7.1 Creating a VM

To create a VM, use the following commands:

```
sudo apt-get install qemu-kvm libvirt-bin bridge-utils virt-manager  
sudo adduser name libvirtd  
sudo virt-manager
```

The virt-manager window allows the creation of a new virtual machine.

1. Click **Create a New Virtual Machine**.

The new VM wizard describes the virtual machine creation process in five steps:

- Naming the guest virtual machine and choosing the installation type
- Locating and configuring the installation media
- Configuring memory and CPU options
- Configuring the virtual machine's storage
- Configuring networking, architecture, and other hardware settings

7.2 L2 Kernel Driver

Use the x86-host-install.sh script. The release source includes the x86-host-install.sh script to extract and build individual components like the L2 driver.

7.3 DPDK

Use the x86-host-install.sh script. The release source includes the x86-host-install.sh script to extract and build individual components like DPDK.

Appendix A: Acronyms and Abbreviations

Table 2 lists the acronyms and abbreviations used in this document.

For a more complete list of acronyms and other terms used in Broadcom documents, go to: <http://www.broadcom.com/press/glossary.php>.

Table 2: Terms and Acronyms

Item	Comment
AP	Application Processor, also referred to as Maia
Cortex-A72	High speed Cortex-A processors from ARM. These cores on Stingray make up main CPUs on the southbound side
DPDK	Data Plane Development Kit
EP	PCIe endpoint
Host	Northbound server host connected via PCIe, Stingray is seen as a PCIe endpoint to the host
Maia	Former codename for Cortex-A72. Marketing/Engineering term for the Cortex-A72
NIC	Network Interface Card
Nitro	Nitro high speed network controller
Northbound	Host side outside of Stingray.
OVS	Open vSwitch, an open-source implementation of a distributed virtual multi-layer switch.
PF	Physical Function
PMD	Poll Mode Driver, a user-mode driver that completely bypasses the kernel
SmartNIC	Inline traffic flow processing by the Maia cores
SoC	System on a Chip
Southbound	ARM SoC side inside Stingray.
SR-IOV	PCIe Single Root I/O Virtualization
VF	Virtual Function

Revision History

5880X-PS225-AN102; May 5, 2020

Updated:

- [Figure 1, Packet Flow Path](#)
- [x86 Side Configuration](#)
- [Figure 2, Setup for VF Pairs](#)
- [Figure 3, Setup for VF Representor Pairs](#)
- [SmartNIC Representor Pairing Model \(with VxLAN tunnel\)](#)

5880X-PS225-AN101; December 14, 2018

Updated:

- MAIA Side Configuration
- OVS (Open vSwitch) Offload Models

5880X-PS225-AN100; October 9, 2018

Initial Release.

